

Cross-LLM Chat Index

v1 Product Specification — Option B (Pure Browser Extension)

Executive Summary

A privacy-first productivity tool that indexes the user's conversations across Claude, ChatGPT, Grok, and Gemini into a single, searchable place. Users continue using each AI tool the way they do today; the extension passively captures their conversations and provides fast search, summaries, and one-click deep-links back to the original thread.

Architecture choice for v1 is **Option B: a pure Chrome/Edge browser extension** with a full-page in-browser UI. No native desktop app, no backend, all data stored locally in the browser. This dissolves the 'web session must be active' sync problem because the extension is alive whenever the browser is.

Why Option B (vs. desktop app or hybrid)

Dimension	Option B (Extension)	Desktop App + Ext.
Build scope	One artifact	Two artifacts
Auto-sync	Passive (always-on)	Requires launch-tab dance
Install friction	One click (Web Store)	Installer + extension
Updates	Automatic via Web Store	Manual / auto-updater
Storage ceiling	IndexedDB (tens of K)	Unlimited (SQLite)
Price ceiling	Lower expectation	Higher expectation
OS packaging	None	Mac + Windows builds

Trade-off accepted: lower per-unit price and a hard ceiling on local storage. Revisit a native wrapper (Option C) only if users hit storage limits or demand a 'real app' feel.

v1 Concrete Plan

Platform & runtime

- Chrome + Edge extension, Manifest V3.
- Firefox added post-v1 if asked for.
- Local-only storage. No backend, no cloud sync in v1.

User interface

- Full-page tab launched from the extension icon (chrome-extension://.../app.html).
- Looks and behaves like a standalone app inside a browser tab.
- Left rail: provider filter (Claude / ChatGPT / Grok / Gemini), date range, tags, starred.
- Main pane: conversation cards — title, provider badge, date, snippet/summary, tags.
- Detail view: full conversation rendered with markdown / code blocks / tables, plus a prominent **'Open in [Provider]'** button.

Storage

- IndexedDB for the conversation index. Comfortable for tens of thousands of conversations.
- Bulk export to JSON file as a manual backup.
- Bulk import to restore on a new machine or browser profile.

Capture (the workhorse)

- Content scripts on **claude.ai**, **chatgpt.com**, **grok.com**, **gemini.google.com**.
- Watch for new or updated conversations as the user chats; write to IndexedDB.
- Capture: title, full message history, conversation URL, timestamps, model used.
- Gemini and Grok are fully covered (web-only).
- Claude and ChatGPT desktop apps are covered *indirectly*: both sync to their web backends, so the extension catches those conversations next time the web app is active.

Backfill & safety net

- Drag-and-drop import of each provider's official export .zip file (ChatGPT, Claude, Gemini via Google Takeout, Grok via X export).
- Parser per provider, intelligent dedupe by conversation ID on re-import.
- 'Add manually': paste a URL + title + note as an escape hatch for anything automation misses.

Search

- v1: keyword full-text search over titles, messages, tags, notes.
- Filters: provider, date range, tag, starred.
- v1.x: optional local semantic search via a small in-browser embedding model (lazy-loaded; ~100–200 MB).

Summaries

- Auto-generate a 1–2 sentence summary per conversation on first index.
- Default: local model running in the extension (free, private, slower).
- Pro option: user supplies a cheap API key (Haiku / GPT-4o-mini / Gemini Flash) for higher quality.

Deep-linking back to the source

- ChatGPT: `chatgpt.com/c/<id>`
- Claude: `claude.ai/chat/<id>`
- Gemini: `gemini.google.com/app/<id>`
- Grok: `grok.com/chat/<id>`
- 'Open in [Provider]' focuses an existing tab if open, else opens a new one.

Organisation

- User-defined tags (e.g. `#client-acme`, `#tax-research`).
- Star / pin important conversations.
- Smart folders — saved searches that auto-update.

Sync status panel

- Per-provider: last successful capture, # of conversations indexed.
- Gentle nudge if a provider hasn't synced in N days: 'Haven't seen Claude conversations in a while — open `claude.ai` to sync.'
- Turns the constraint into a normal background-sync UX, not a mysterious gap.

Explicitly Out of Scope for v1

- No prompt engine — users keep using Claude / ChatGPT / Grok / Gemini directly.
- No editing of original conversations.
- No multi-device real-time sync (v2).
- No team or sharing features.
- No automatic model failover or routing.
- No iOS or Android client (v2: read-only viewer + Share Sheet target).
- No native desktop wrapper (revisit only if storage or pricing forces it).

Provider Capture Matrix

Provider	How user accesses it	v1 capture path
ChatGPT	Web + desktop app + iPhone	Web extension; desktop syncs via web; iPhone via export or Share
Claude	Web + desktop app + iPhone	Web extension; desktop syncs via web; iPhone via export or Share
Gemini	Web only (no desktop app)	Web extension (full coverage)
Grok	Web only (grok.com / x.com)	Web extension (full coverage)

Expectation to document explicitly in onboarding: the extension captures ChatGPT and Claude *desktop app* conversations only when the user's web session is active in the same browser. In practice this means keeping a pinned tab open, or visiting the web version occasionally. The Sync Status panel surfaces this naturally.

Risks & Sticking Points

ToS grey area for the extension.

Reading the user's own conversations from their own logged-in session is generally tolerated. Ship clear language: 'The extension reads only your own conversations from your own browser, never sends them to us.' Local-only storage strengthens this position.

DOM breakage on provider redesigns.

Every web UI redesign at Claude / ChatGPT / Gemini / Grok can break content-script selectors. Budget ~2–4 hours/month of maintenance. Manageable but ongoing.

iPhone coverage gap.

v1 captures iOS conversations only after they sync to the provider's web backend (usually seconds), and only when the user next has the web app active. Mobile-first capture (Share Sheet target, native viewer) is a v2 item. State this clearly on the landing page.

IndexedDB storage limits and purging.

Browsers can evict IndexedDB under storage pressure. Mitigate with the navigator.storage.persist() API and regular export-to-file backups.

Provider export lag.

ChatGPT exports can take hours to arrive. Acceptable for backfill, unacceptable as the only ingestion path. Extension is the live path.

Search quality on long technical threads.

Keyword FTS handles most queries; code-heavy or mixed-language threads sometimes need semantic search. Plan local embeddings as a v1.x add-on.

Competition window is open but narrow.

Nothing dominant in the multi-LLM cross-provider search niche today. A 6–12 month window before someone bigger ships something similar.

Pricing & Positioning

- **Free tier:** manual .zip imports, keyword search, deep-link.
- **Pro (one-time \$19.99–\$29 or \$4–5/month):** auto-capture, semantic search, summaries, smart folders.
- **Distribution:** Chrome Web Store + a marketing site selling license keys via Stripe. Keeps the customer relationship.
- **Positioning line:** ‘All your AI conversations, searchable in seconds. Stays on your machine.’
- **Audience to seed:** Hacker News, productivity Twitter, finance / consulting / dev subreddits, LinkedIn AI groups.

v1 Build Order

- 1 Extension scaffolding (MV3, build pipeline, packaging script).
- 2 IndexedDB schema + data-access layer.
- 3 Importer for ChatGPT export .zip (best-documented format).
- 4 Full-page UI shell + browse/search list view.
- 5 FTS5-equivalent keyword search over IndexedDB.
- 6 Detail view + ‘Open in Provider’ deep-link.
- 7 Content script for ChatGPT — prove the live-capture pattern end-to-end.
- 8 Extend content scripts to Claude, Gemini, Grok.
- 9 Importers for the other three providers’ export formats.
- 10 Sync status panel + storage persistence prompt.
- 11 Tags, stars, smart folders.
- 12 Local-model summaries.
- 13 Polish, license-key flow, Chrome Web Store submission.

Rough effort: 6–10 weeks of focused evening/weekend work for a competent solo developer; longer if new to MV3 extensions.

Open Decisions Before Build Starts

- **Pricing model:** one-time vs. subscription.
- **Licence delivery:** Chrome Web Store payments vs. external Stripe + licence key.
- **Summary engine default:** local model from day one, or require API key for Pro only.
- **Brand & name:** Croftlands sub-brand vs. standalone product brand.

- **Beta cohort:** private list of 20–50 multi-LLM users before public launch.